

区块链基础 - Lecture Interlude

插播课程：关于定义、定理和证明 - Interlude: Who Cares About Definitions, Theorems, and Proofs?

笔者按：无论你是硕博学生，研究人员，计算机科学家，还是区块链领域的程序员，对于这样的计算机科学基础——定义、定理和数学证明，都是你应该掌握的基本功。

假如你是区块链领域的前面三类人员，毫无疑问的，这些基础的内容是必须要掌握的。如果你是区块链开发者，想要成为行业顶尖的开发者，这也是必须掌握的内容。否则你只能在已有的基础设施（并且当成一个黑盒）上做开发，无法从根本上理解共识协议的能力、潜力和局限性，以及一些基本的密码学原语、基础的博弈论和激励机制等知识，这会使你的创造力在很大程度上受限。没有这些理论基础，几乎没有可能看懂论文、了解最新进展，即便有AI的解读也不能。

本系列课程中引入数学，并不是想让理论显得高大上，而是使得这些理论能够被足够精确地描述出来，并且更容易理解（是的，通常被认为难以理解的数学，反而会让理论更容易被理解，因为精确）。不要惧怕数学，尝试理解它，运用它，多多练习证明技巧，学习证明结构，是可以熟能生巧的。学会这个技能，会让你对理论的理解扎根在泥土里，练习的越多，根茎就越粗壮，让你汲取更多的养分，有更多的机会和能量去释放你的创造力，做出更有创意、更实用、更安全的产品。

数学 >> 直觉与希望 - Math >> Intuition and Hope

本节标题是 数学 远大于 直觉与希望。意为有数学证明支持的共识协议，远比仅仅靠直觉和白皮书里的几句话给你的希望要靠谱得多。

建议：如果你是一位区块链领域的开发人员/工程师，你正在比较不同的共识解决方案，请务必选择有数学证明支持的方案，这比白皮书或自媒体中宣传的寥寥几笔写出来的直觉和希望要靠谱得多。至少这个方案应该具备有精确描述假设下的你希望具备的属性的证明。

例如，你正在使用某种密码学签名方案，这种签名方案所使用的假设是：离散对数问题是困难的。你必须清楚这个假设的存在，以及在该假设下，这种方案具备了你想要的性质的证明。如果假设不成立，那么这个方案就无法正常工作，这是作为工程师应该要清楚的。

经验：如果在某种方案中，它宣称自己满足某种你想要的属性，但没有数学证明，那就需要对它持怀疑态度，这种方案可能是有某些未知缺陷的。

这个经验并不总是正确，或许某种解决方案一开始没有数学证明，人们并不清楚在数学上这种方案的有效性能达到什么程度。直到大家用这个方案好几年之后才搞清楚在数学上要如何证明。这种情况是存在的，但不多。而没有数学证明的方案在后来发现因存在缺陷而被淘汰，这种情况更常见一些。但也存在一种情况是，某种解决方案在数学上可能存在缺陷，但并不妨碍它的实际用途，例如BGP路由协议。

但这个经验在分布式系统领域，往往比较奏效。在分布式系统的相关协议中，即便协议代码发生一些非常微小的改动，也有可能破坏其数学性质，从而导致某些我们需要的性质不成立，例如我们已经很熟悉的活性和一致性。更为常见的一种情况是，在设计协议的时候，比如白皮书/论文中证明了协议的一些性质，但具体代码的实现和论文中有所偏差，或者存在bug，导致这些性质被破坏。

有没有可能，在理论中，一种边界情况导致某种解决方案的某个性质的不成立，但现实中几乎不会出现这种边界情况？

有。但这种可能性较低，具体问题具体分析。我们当然可以用测试网的方式来检验一种分布式协议是否工作良好，但如果这个分布式协议之上运行了高价值的项目，只是通过测试网来检验或许是远远不够的，这种检验无法覆盖主网中所有可能出现的用例。即便这个协议在主网上线后运行了一年了，没什么问题发生，作为一位严谨的工程师，依然会不够放心，随时担心可能发生什么。用数学证明的方式更靠谱。

定义、定理和证明 - The Three Pillars of Math

定义 - 让你清楚你在表达什么，让别人也清楚你所表达的内容，基于定义进行讨论和辩论，避免各说各的，不在一个频道上。

例：

- 被滥用的一个概念“decentralization - 去中心化”。经常会见到有人会争辩，哪个区块链更加“去中心化”，哪个不够“去中心化”。但人们所争辩的“去中心化”是同一回事吗？一个更加精确的定义能够让我们的讨论更加有效。

定理 - 被断言为真的形式化的数学命题，有证明作为支持。

例：

- 可能性结果（possibility results）阐明了在某些假设下，一个解决方案具备我们所需的属性 - articulates assumptions under which solution has desired properties. 可能性结果类定理让我们在设计一种系统的时候能够放心大胆地使用某种解决方案，因为它的背后有数学证明的支持。如果你使用了某种解决方案，但事实上它不具备我们想要的属性，从定理和数学证明中也能够看出问题出在哪里，比如是否假设条件被打破，是否代码实现没有按照定理的要求来，出现了bug？这样我们就很清楚下一步该做什么。
- 不可能性结果（impossibility results）：阐明了在某些假设下，不可能存在一种满足某些性质的解决方案。如果不知道这些不可能性结果，你可能会浪费大量时间在不可能的事情上。不可能性结果并不总是令人沮丧的，人们可以通过某些方式绕过不可能性结果，比如牺牲某一些性质，加强在工程实践中更加重要的性质，或者改变假设条件让不可能变成可能。

证明 - 证明阐释了某个命题为何为真，并在工程实践中给予我们信心。实际上，在证明的过程中，我们往往可以从中推理出一个解决方案。不仅工程是实践，学术也是实践，证明也是实践。自己尝试进行证明和阅读证明，能够让你更深刻地理解某个定理，更快速地理解某个定理在修改一些陈述或假设条件时是否仍然成立，以及更快速地定位工程实践中出bug的地方。例如，在Tendermint协议中，如果 $f = 34$ 而不是33，只是增加了1个拜占庭节点而已，有认真学习过证明的同学能够很快想到，此时出现了两个超级多数投票群体，拜占庭节点可以对区块进行双重投票，从而破坏一致性。